StepWise: Validation Testing, Result Analysis and Discussion (Review-3)

Siddhi J Salunkhe
23BDS0243

Sanvi Saket
23BDS0210

Abstract—StepWise is an explainable, CPU-only tutor that reads handwritten arithmetic and linear-equation working, verifies each step symbolically, names the misconception and adapts practice. This Review-3 paper reports its validation testing, result analysis and discussion. Each learned module was validated against baselines on held-out data: the HOG + SVM symbol recognizer reaches 92.6% symbol accuracy (macro-F1 0.899) on the official CROHME 2014/2016/2019 test sets written by unseen writers; a PFA + IRT + recency model predicts the next answer of real ASSISTments 2009 students with AUC 0.815; the Random Forest mistake classifier attains macro-F1 0.994 on synthetic solutions against 0.330 for the earlier rule-based system; the adaptive practice policy masters 74.1% of skills in 40 items against 63.8% for random selection; and K-Means recovers planted student groups with ARI 0.82. Functional validation is provided by an automated suite of 143 unit, integration and API tests. We discuss why real data changed model choices, where the results are upper bounds, and the remaining threats to validity.

Index Terms—handwritten mathematics recognition, support vector machine, knowledge tracing, item response theory, mistake diagnosis, intelligent tutoring systems, validation testing

I. Introduction

StepWise is built and every learned module beats its baseline on held-out data; this review shows how that was tested and what the numbers mean. Reviews 1 and 2 covered the problem statement, literature survey, architecture and implementation. Review-3 covers validation testing, result analysis and discussion.

The system takes a photo, camera capture or on-screen drawing of handwritten working. It runs an 11-stage pipeline: OpenCV clean-up and segmentation, a symbol recognizer (M1), SymPy step verification [11], a mistake classifier (M2), knowledge tracing with an IRT recommender (M3) and class grouping (M4). Student corrections to misread symbols feed an active-learning loop. All learned parts are classical machine learning (scikit-learn [10], NumPy, SciPy), run offline on a laptop CPU and total about 6 MB.

The review answers four validation questions:

1) Does the symbol recognizer read real handwriting from writers it has never seen?
2) Does knowledge tracing predict real students' next answers better than simple baselines?
3) Does the learned mistake classifier diagnose misconceptions better than the earlier rule-based system?
4) Do the adaptive practice policy and the class grouping do what they claim on controlled data?

Section II describes the validation protocol and test suite. Section III gives the setup. Section IV reports and analyses results. Section V discusses them and Section VI concludes.

II. Validation Testing

Validation has two layers: model validation on held-out data with no leakage, and functional validation by an automated test suite.

A. Model Validation Protocol

Each module uses a split that matches how it will meet new data in use. Model and hyperparameter choices are made only on validation data; test sets are touched once (Table I).

Three safeguards prevent optimistic results. First, CROHME test writers never appear in training. Second, the fonts-only model is kept as a control to show the effect of real data. Third, every learned model is compared with at least one non-learned or simpler baseline (rules, EMA recency, random policy).

B. Label Validation

CROHME labels whole expressions, not symbols, so symbol labels come from count alignment: the app's segmenter runs on each linear expression, and crops are labelled left to right only when the symbol count equals the LaTeX token count. A manual check of 80 random crops found 79 correct (98.75%).

C. Functional Validation (Test Suite)

The backend has 143 automated pytest cases (counted after parametrization) across 13 test modules (Table II). They cover four types: unit tests of each feature and metric, model tests that check a learner recovers known parameters, integration tests of the full pipeline, and API tests of each user flow. The frontend is validated by a successful production build.

III. Experimental Setup

All experiments run on a laptop CPU with a fixed random seed (0); the full training run takes about 8 minutes with real data.

TABLE I
Validation Protocol per Module

Module	Training data	Validation (for choices)	Test (reported)	Split rule
M1 Symbol recognizer	CROHME train (6,271 symbols) [1], HASYv2 (3,335 symbols, 144 writers) [2], 12,210 font glyphs	15% held-out CROHME-train expressions + 25% held-out fonts; mean macro-F1	Official CROHME 2014/16/19 test: 1,821 symbols, 127 expressions	Writer-disjoint; font-disjoint
M2 Mistake classifier	2,520 synthetic solutions	5-fold stratified CV on the 80%	630 synthetic solutions (70 per class)	Stratified 80/20
M3 Knowledge tracing (real)	ASSISTments 2009 [7], 80% of 4,151 students	Validation students from training split (Rasch L2)	74,867 next-answer predictions	Learner-disjoint 80/20
M3 Knowledge tracing (sim.)	600 simulated learners, 93,671 attempts	–	Held-out learners	Learner-disjoint 80/20
M3 Recommender policy	Simulated learners	–	40 items per learner, 5 seeds	Repeated simulation
M4 Class grouping	120 simulated students, 4 planted archetypes	k by silhouette (2–6)	ARI vs planted groups	Unsupervised

TABLE II
Automated Test Suite

Module	Tests	Validates
learning_models	16	BKT/Rasch recovery, recommender, clustering
api_endpoints	13	Submission, progress, quiz endpoints
math_parser	13	SymPy parsing without eval
ml	12	Symbol and 33 mistake features; mini training run
benchmark_metrics	11	CER, WER, exact match, macro-F1
ml_api	11	ML Lab, correction, retraining, registry
quiz_service	11	Item bank answers verified with SymPy
providers	8	ML, demo and optional OCR providers
diagnosis	7	Step-error localisation, guardrails
upload_validation	7	File type, size, corrupt images
database_operations	5	Persistence of students, mastery
benchmark_service	3	Benchmark summary over a manifest
preprocessing	3	OpenCV clean-up stages

A. Environment

Python 3.10+, scikit-learn 1.5.2, NumPy 1.26.4, SymPy 1.13.2, OpenCV 4.10, FastAPI 0.115 and SQLAlchemy 2.0 on the backend; React with Vite on the frontend. No GPU, neural network or API key is used. Every number in Section IV is regenerated by the training script and stored as JSON reports.

B. Models and Hyperparameters

- M1 (374 features: HOG [3] 324 + 7×7 zoning 49 + log aspect 1): SVM [4] (RBF, $C = 10$, γ = scale); Random Forest (200 trees); Logistic Regression ($C = 0.5$); kNN ($k = 5$, distance-weighted).
- M2 (33 named features): Random Forest [9] (300 trees, min leaf 2); HistGradientBoosting (200 iterations, lr 0.1); Decision Tree (depth 8); Logistic Regression ($C = 1$); rule-based baseline.
- M3: EMA, BKT [5] (EM per skill), PFA [6], PFA + Rasch IRT [8], PFA + IRT + recency; Rasch L2 chosen from $\{0.05, 1.0, 5.0\} \rightarrow 1.0$.
- M4: K-Means (k from 2–6 by silhouette, $n_{init} = 10$) vs agglomerative (Ward).

C. Evaluation Metrics

Classification (M1, M2) uses accuracy, macro-F1 and confusion matrices; expression reading uses exact-match rate and character error rate (Levenshtein distance / reference length); next-answer prediction (M3) uses ROC AUC, RMSE and log-loss; the policy uses share of skills mastered after 40 items; clustering uses adjusted Rand index (ARI) [13], silhouette and Davies–Bouldin. Macro-F1 is the main classification metric because small classes such as '/' (9 test symbols) would be hidden by plain accuracy:

$$\text{Macro-F1} = \frac{1}{K}\sum_{k=1}^{K}\frac{2P_kR_k}{P_k + R_k}. \tag{1}$$

IV. Results and Result Analysis

Every learned module beats its baseline on held-out data; the largest single gain is real training data, which lifts symbol accuracy on unseen writers from 55.3% to 92.6% (Table III).

A. M1: Symbol Recognition on Real Handwriting

Table IV shows the same SVM trained on three datasets. The fonts-only model scores 0.975 macro-F1 on held-out fonts but 0.573 on real writers, so printed fonts are a poor stand-in for handwriting. Real-only and fonts + real are tied on handwriting; fonts + real was chosen on validation (0.930 vs 0.899) because it also reads printed worksheets, where real-only misreads some printed '4's.

Validation ranking matches test ranking for the winner (Table V), which supports the selection protocol. Results are stable across years: 91.1% (2014, $n = 461$), 93.1%

TABLE III
Headline Results (Seed 0)

Module	Data	Selected model	Test result	Best baseline
M1 Symbol recognizer	Real (CROHME test, unseen writers)	SVM (RBF)	92.6% acc., macro-F1 0.899	Fonts-only training: 55.3%
M3 Knowledge tracing	Real (ASSISTments 2009)	PFA + IRT + recency	AUC 0.815	EMA: 0.811
M2 Mistake classifier	Synthetic	Random Forest	Macro-F1 0.994	Rule-based: 0.330
M3 Practice policy	Simulated	Adaptive (BKT + IRT)	74.1% skills mastered	Random: 63.8%
M4 Class grouping	Simulated	K-Means ($k = 5$)	ARI 0.82	Agglomerative agrees at ARI 0.86

TABLE IV
Effect of Training Data (SVM, CROHME Test)

Training data	Acc.	Macro-F1	Expr. exact	CER
Fonts only	0.553	0.573	0.220	0.452
Real only	0.927	0.913	0.575	0.120
Fonts + real	0.926	0.899	0.575	0.110

TABLE V
Algorithm Comparison (Fonts + Real Training)

Algorithm	Val. score	Test acc.	Test F1	Fit (s)
SVM (RBF)	0.930	0.926	0.899	32.8
Logistic Regression	0.911	0.903	0.880	–
Random Forest	0.916	0.900	0.875	6.7
kNN	0.907	0.878	0.848	0.0

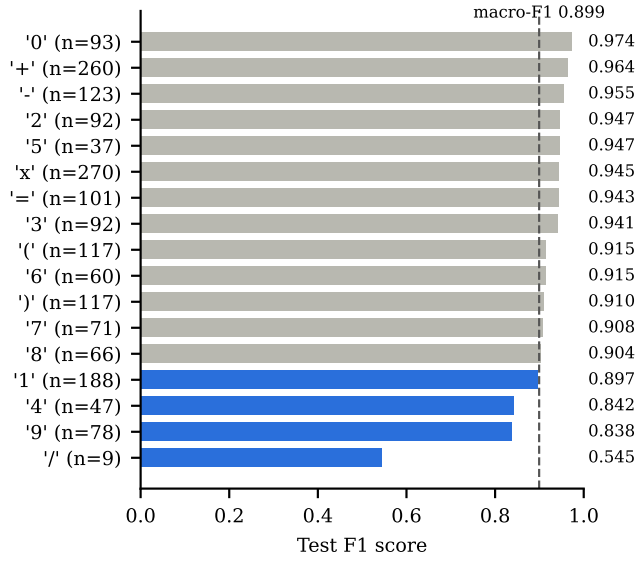


Fig. 1. Per-class test F1 of the SVM symbol recognizer on CROHME 2014/16/19. Blue: classes below the macro average (dashed).

(2016, $n = 666$) and 93.1% (2019, $n = 694$). Fig. 1 shows that 14 of 17 classes reach F1 $\geq$ 0.89; the slash is the outlier at 0.545, but it has only 9 test samples.

Error analysis. The most frequent confusions are '1' read as '/' (12 of 188), 'x' as '1' (7), '=' as '−' (5) and '9' as '3' (5). All are shape-similar pairs: a slanted '1' resembles a slash, and an '=' whose bars are split by segmentation becomes a minus. Symbol accuracy of 92.6% gives only 57.5% exact expressions because one wrong symbol spoils the whole line: with about 4 symbols per aligned expression (1,821 symbols from 468 expressions), independent errors alone give $0.926^4 \approx 0.74$, and segmentation errors lower it further.

TABLE VI
Mistake Classifier (630 Held-out Synthetic Solutions)

Algorithm	CV macro-F1	Test F1	Fit (s)
Random Forest	0.994 ± 0.004	0.994	0.22
Logistic Regression	0.990 ± 0.005	0.992	0.04
Gradient Boosting	0.988 ± 0.002	0.991	3.57
Decision Tree	0.971 ± 0.007	0.979	0.00
Rule-based (previous)	–	0.330	–

TABLE VII
Next-Answer Prediction, ASSISTments 2009 (74,867 Predictions)

Model	AUC	RMSE	Log-loss
PFA + IRT + recency	0.815	0.396	0.476
EMA (recency baseline)	0.811	0.403	0.494
PFA	0.787	0.404	0.490
PFA + Rasch IRT	0.781	0.413	0.525
BKT (EM, per skill)	0.771	0.408	0.513

B. M2: Mistake Classification

The Random Forest makes 4 errors in 630 (Table VI): one arithmetic slip labelled as moving terms, one moving-terms error labelled as an arithmetic slip, one unreadable sample labelled correct, and one insufficient-information sample labelled unreadable. The rule-based system (accuracy 0.410), by contrast, sent 27 of 70 arithmetic slips and 33 of 70 sign errors to "final answer mismatch": it saw that the answer was wrong but could not say why. The most important features are final_matches_expected (0.100), ocr_confidence (0.097), n_equations (0.075) and rel_error (0.058).

TABLE VIII
Practice Policy (40 Items per Learner, 5 Seeds)

Policy	Skills mastered	Wasted practice
Adaptive (BKT + IRT)	0.741 ± 0.008	28%
Round-robin	0.640 ± 0.010	47%
Random	0.638 ± 0.013	49%

C. M3: Knowledge Tracing and Recommendation

On simulated learners the ranking is reversed: BKT leads (AUC 0.835) ahead of PFA (0.832) and EMA (0.820). BKT recovers the simulator's true parameters with mean absolute error 0.021, and Rasch difficulties correlate 0.75 with true difficulty.

The adaptive policy, which picks the least-mastered skill at about 70% predicted success, masters 10.3 percentage points more skills than random and wastes 21 points less practice on already-learned skills (Table VIII).

D. M4: Class Grouping

On 120 simulated students with 4 planted archetypes, silhouette selected $k = 5$. K-Means recovers the planted groups with ARI 0.82 (silhouette 0.245, Davies–Bouldin 1.43) and agrees with agglomerative clustering at ARI 0.86. Two of the five clusters carry the same name, "Struggling with moving terms & skipped steps", suggesting the extra cluster splits that archetype.

V. Discussion

The results support the main design choice: classical, explainable models are accurate enough for a CPU-only tutor when they are trained and tested on real data.

A. Real Data Changes the Conclusions

Two findings would have been missed without real data. First, the fonts-only recognizer looked excellent on held-out fonts (0.975) and failed on real writers (0.573); synthetic validation alone would have shipped a weak model. Second, on simulated learners BKT is the best tracer, but on real ASSISTments students it is the worst of five (AUC 0.771). Skill-builder sets end after three correct answers in a row, so recent answers are highly predictive; adding recency features to PFA gives the best model (0.815).

B. Why the SVM Wins

The RBF SVM leads on both validation and test, ahead of Random Forest by 2.6 points of accuracy. HOG features describe stroke orientation in local cells, and the RBF kernel separates shape-similar classes smoothly in that space. Its cost is speed: 0.43 ms per symbol against 0.009 ms for Random Forest, still negligible for a page of 10–40 symbols.

C. Explainability

The mistake classifier works from 33 named features, so each diagnosis shows the evidence that fired, the class probability and the runner-up. Top features match how a teacher reasons: is the final answer right, could the work be read, and how many equations were written. Guardrails return "couldn't read clearly" when OCR confidence is below 0.35 rather than guess.

D. Comparison with Deep-Learning OCR

TrOCR [12], Donut and vision-language APIs remain optional baselines in the benchmark module, but no head-to-head numbers on CROHME are reported yet. The classical pipeline's advantages are offline use, about 6 MB of models and per-symbol confidence that drives active learning; its accuracy against TrOCR on the same test set is still open.

E. Limitations and Threats to Validity

- Synthetic mistake data. No public dataset of handwritten solutions with misconception labels exists, so M2's 0.994 is an upper bound.
- Image domain. CROHME is tablet handwriting rendered to images, not phone photos of paper.
- Label noise. Count alignment mislabels about 1 crop in 80.
- Skill mismatch. ASSISTments skills approximate StepWise's 8 skills; S8 has no match.
- Simulator bias. The simulator is BKT + IRT-like, so the policy study favours BKT-based policies.
- Segmentation. Touching or cursive symbols are under-segmented; scope is arithmetic and linear equations.
- Single seed. Most results use seed 0; only the policy study reports 5 seeds.

VI. Conclusion and Future Work

Validation shows StepWise is ready for a classroom pilot: it reads unseen writers' symbols at 92.6% accuracy, predicts real students' next answers at AUC 0.815, and every learned module beats its baseline. The mistake classifier triples the rule-based macro-F1 (0.994 vs 0.330) on synthetic data, and the adaptive policy masters 74.1% of skills against 63.8% for random practice. 143 automated tests validate the features, models and user flows.

Future work, in priority order: (1) hand-label 100–200 real photographed solutions to test M2 and the full pipeline; (2) benchmark against TrOCR on CROHME test expressions; (3) fit item difficulties on StepWise's own practice logs; (4) learned region and component-merge classifiers for touching symbols; (5) report mean ± standard deviation over several seeds.

References

[1] H. Mouchère, C. Viard-Gaudin, R. Zanibbi, and U. Garain, "ICFHR2016 CROHME: Competition on recognition of online handwritten mathematical expressions," in Proc. 15th Int. Conf. Frontiers in Handwriting Recognition, 2016, pp. 607–612.

[2] M. Thoma, "The HASYv2 dataset," arXiv:1701.08380, 2017.
[3] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in Proc. IEEE CVPR, 2005, pp. 886–893.
[4] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.
[5] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," User Modeling and User-Adapted Interaction, vol. 4, no. 4, pp. 253–278, 1994.
[6] P. I. Pavlik, H. Cen, and K. R. Koedinger, "Performance factors analysis – A new alternative to knowledge tracing," in Proc. 14th Int. Conf. Artificial Intelligence in Education, 2009, pp. 531–538.
[7] M. Feng, N. Heffernan, and K. Koedinger, "Addressing the assessment challenge with an online system that tutors as it assesses," User Modeling and User-Adapted Interaction, vol. 19, no. 3, pp. 243–266, 2009.
[8] G. Rasch, Probabilistic Models for Some Intelligence and Attainment Tests. Copenhagen: Danish Institute for Educational Research, 1960.
[9] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
[11] A. Meurer et al., "SymPy: Symbolic computing in Python," PeerJ Computer Science, vol. 3, e103, 2017.
[12] M. Li et al., "TrOCR: Transformer-based optical character recognition with pre-trained models," in Proc. AAAI, 2023, pp. 13094–13102.
[13] L. Hubert and P. Arabie, "Comparing partitions," J. Classification, vol. 2, no. 1, pp. 193–218, 1985.